

Database Connection String in Entity Framework Core

In this article, I will discuss **Database Connection String in Entity Framework Core** and **How to Generate a Database using the EF Core Code First Approach**. Please read our previous article discussing the [DbContext Class in Entity Framework Core](#). We will work with the example we created in our previous articles.

What is a Database Connection String in Entity Framework Core?

A database connection string is a string that specifies information about how to connect to a particular database. In the context of Entity Framework Core (EF Core), a connection string provides the necessary details for the DbContext to establish a connection to the database. A typical connection string includes:

- **Server Name:** The name or network address of the database server.
- **Database Name:** The name of the specific database to which to connect.
- **Credentials:** Username and password for authenticating to the database.
- **Other Settings:** Optional parameters like timeout settings, encryption settings, and more.

The connection string is essential for establishing a connection between your application and the database, enabling Entity Framework Core to execute queries and commands against the database.

Different Mechanisms to Store Database Connection String in Entity Framework Core

Now, we will see the options available in .NET Core to Provide the Database Connection String. We can provide the database connection string to the EF Core application in several ways. The connection strings were stored in the web.config file in the older version of .NET Framework Applications. The newer .NET Core applications can read the database connection string from various sources. They are as follows:

- **Code-Based Configuration:** The connection string can be directly specified in the OnConfiguring method of the DbContext class.
- **appsettings.json:** The connection string is stored in the appsettings.json file, a central configuration file for .NET Core applications.
- **Environment Variables:** For security reasons, the connection strings can be stored in environment variables commonly used in cloud environments.
- **Secret Manager:** The Secret Manager tool can securely store sensitive information in the development environment, such as connection strings.
- **Command-Line Arguments:** When the application starts, the connection string can be passed as a command-line argument.

Providing Connection String in OnConfiguring Method of DbContext Class:

Please modify the DbContext Class as follows to configure the connection string by overriding the OnConfiguring Method of the DbContext class. Here, we are removing the Constructor as we are providing the connection string from the OnConfiguring method.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    // EFCoreDbContext is your custom DbContext class that extends the base DbContext by EF Core.
    public class EFCoreDbContext : DbContext
    {
        // The OnConfiguring method allows us to configure the DbContext options.
```

```

// such as specifying the database provider and connection string.
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    // Configuring the connection string to use a SQL Server database.
    // UseSqlServer is an extension method that configures the context
    // to use a SQL Server database.
    optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB1;Trusted_Connection=True;TrustServerCertificate=True");
    // DbSet<Student> corresponds to the Students table in the database.
    // It allows EF Core to track and manage Student entities.
    public DbSet<Student> Students { get; set; }
    // DbSet<Branch> corresponds to the Branches table in the database.
    // It allows EF Core to track and manage Branch entities.
    public DbSet<Branch> Branches { get; set; }
}
}

```

Code Explanations:

DbContextOptionsBuilder: This is used to configure the options for the DbContext, like which database provider to use and how to connect to the database.

UseSqlServer Method: Since we use SQL Server as the database provider, UseSqlServer is an extension method provided by the SQL Server provider package for EF Core. This method configures the DbContext to use SQL Server and requires a connection string as a parameter.

Connection String Components:

- **Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV:** Specifies the database server's name or network address.
- **Database=EFCoreDB1:** Indicates the name of the database to connect to or create if it doesn't exist yet.
- **Trusted_Connection=True:** Enables Windows Authentication mode, which uses your Windows credentials to connect to the database.
- **TrustServerCertificate=True:** Instructs the connection to trust the server's SSL certificate without requiring validation of the certificate's chain of trust. This is often used in development environments.

EF Core will use this connection string to create a database when we run the migration. So, we have created the entities and DbContext class and specified the Database Connection String for the DbContext. Now, it's time to generate the database and tables and perform the database operations.

How to Add Entity Framework Core Migration

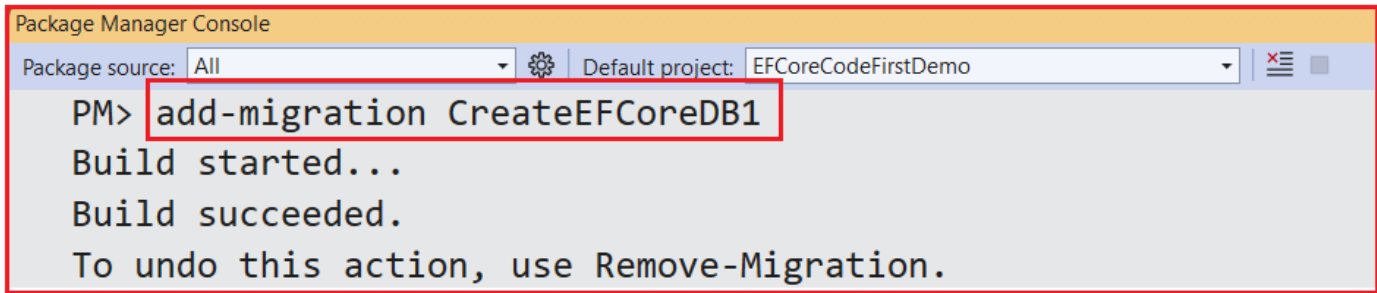
Migrations in EF Core are a mechanism to keep your database schema in sync with your EF Core model. As your application evolves, your data model may change, requiring corresponding changes to the database schema. Migrations help manage these changes over time without losing data or creating inconsistencies.

First, ensure that the EF Core tools package is installed in your project. If not, you can install it via the NuGet Package Manager Console using the following command:

[Install-Package Microsoft.EntityFrameworkCore.Tools](#)

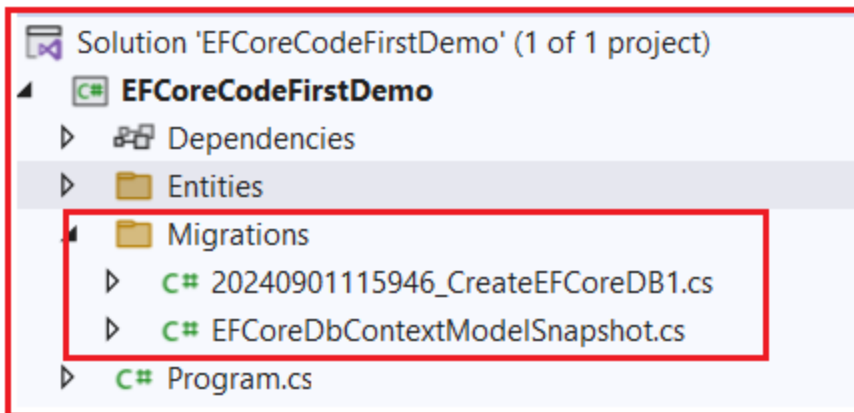
Creating Migration:

To create a migration, we need to use the Package Manager Console in Visual Studio. To Open it, go to **Tools => NuGet Package Manager => Package Manager Console**. This will open the Package Manager Console. Now, type the **Add-Migration CreateEFCoreDB1** command, select the project where your Context class is, and press the enter button, as shown in the image below. Here, **CreateEFCoreDB1** is the name of the migration. You can name it anything that describes the changes (e.g., AddStudentEntity, UpdateBranchTable, etc).



```
Package Manager Console
Package source: All [v] [g] Default project: EFCoreCodeFirstDemo [v] [x] [ ]
PM> add-migration CreateEFCoreDB1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
```

Once the above code is executed successfully, a new folder named Migrations will be created for your project, as shown in the image below. This folder contains the files that EF Core generates to apply changes to the database schema.



Migrations Files:

EF Core generates two primary files in the Migrations folder when we run the Add-Migration command.

Migration Class File (20240901115946_CreateEFCoreDB1.cs): This file contains the Up and Down methods that define the schema changes needed to apply or revert the migration.

- **Up Method:** Contains the code to apply the changes, such as creating or modifying tables and columns.
- **Down Method:** Contains the code to revert these changes, essentially undoing what the Up method did.

Model Snapshot File (EFCoreDbContextModelSnapshot): This file contains a snapshot of your model's structure when the migration was created. EF Core uses it to compare the model's current state with its state in previous migrations, helping EF Core determine what changes need to be applied in future migrations. It ensures that only the necessary changes are made to the database schema.

What Happens When You Execute Add-Migration Command in EF Core?

The Add-Migration command generates a migration based on the current state of your EF Core model. Here's what happens when we execute this command:

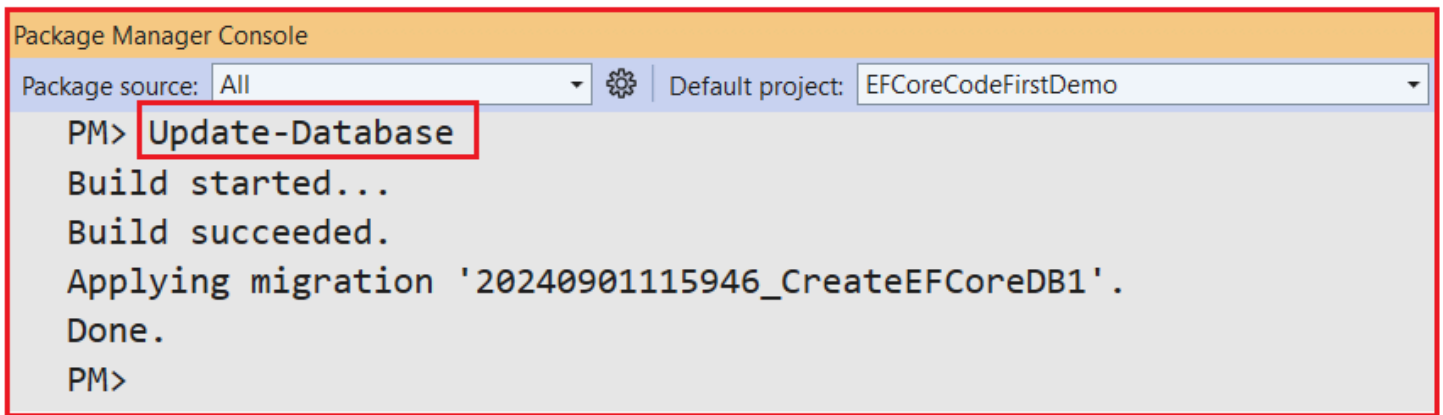
- **Model Snapshot Creation:** EF Core creates a snapshot of your current model state. This snapshot compares your model against the database schema to determine the changes that need to be applied. The snapshot is

stored in a file within the Migrations folder.

- **Migration Class Generation:** EF Core generates a new migration class in the Migrations folder. This class contains two key methods:
 1. **Up Method:** Specifies the operations (like creating tables and adding columns) that need to be applied to bring the database schema sync with the model.
 2. **Down Method:** This specifies how to revert the changes made by the Up method. It is useful if you need to roll back a migration.
- **Code Generation:** The migration class includes all the necessary SQL commands to create, alter, or drop database objects (tables, columns, constraints, etc.) based on the changes detected in your model.

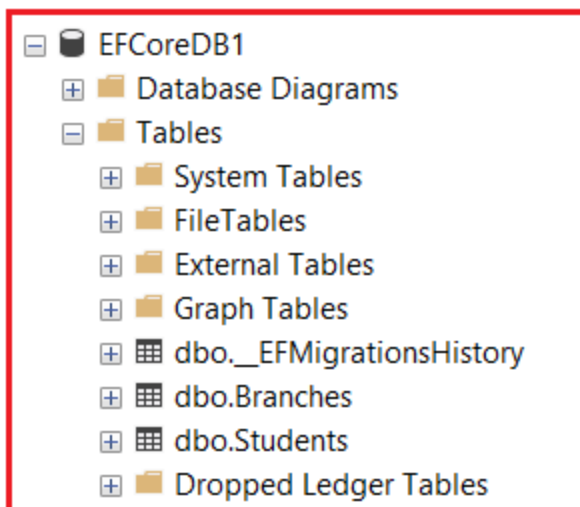
Update Database

After creating a migration, don't assume that your database is automatically updated. The migration files are generated. We need to sync our code base with the database using the **Update-Database** command in the Package Manager Console. So, open the Package Manager Console and then execute the **Update-Database** command as follows:



```
Package Manager Console
Package source: All | Default project: EFCoreCodeFirstDemo
PM> Update-Database
Build started...
Build succeeded.
Applying migration '20240901115946_CreateEFCoreDB1'.
Done.
PM>
```

Once the above command is executed successfully, the database will be created with the name and location specified in the connection string. If you verify the database, it should create the EFCoreDB1 database, a table for each DbSet property (Students and Branches), as shown in the image below.



What Happens When You Execute Update-Database?

- **Applying Migrations:** EF Core looks for any pending migrations (those not yet applied to the database) and runs the Up methods in the migration files sequentially.
- **Database Schema Update:** The changes defined in the Up method, such as creating or modifying tables, are applied to the database, ensuring the schema matches the current model.

- **Migration History Table Update:** EF Core updates a special table in the database called `__EFMigrationsHistory`, which tracks all the migrations that have been applied. This table helps EF Core determine which migrations are pending and need to be applied.

Note: The most important point to remember is that whenever we add or modify domain classes or configurations, we need to sync the database with the model using the **Add-Migration** and **Update-Database** commands. Each time we generate the Migration, we need to provide a name that should have been provided earlier.

Performing Database CRUD Operations using EF Core DbContext Class:

Now, we can use the DbContext class, i.e., `EFCoreDbContext`, to perform database CRUD operations using Entity Framework Core. To better understand this, please modify the Program class as follows: The following example code is self-explained, so please read the comment lines for a better understanding.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // Create an instance of the DbContext class
            using var context = new EFCoreDbContext();
            // Adding two new Branches
            AddBranches(context);
            // Adding two new Students
            AddStudents(context);
            // Retrieve and display all students
            GetAllStudents(context);
            // Retrieve and display a single student by ID
            GetStudentById(context, 1); // Assuming 1 is the StudentId
            // Update a student's information
            UpdateStudent(context, 1); // Assuming 1 is the StudentId
            // Delete a student by ID
            DeleteStudent(context, 2); // Assuming 2 is the StudentId
            // Final retrieval to confirm operations
            GetAllStudents(context);
            Console.WriteLine("All operations completed successfully!");
        }
        private static void AddBranches(EFCoreDbContext context)
        {
            // Create two new Branch objects
            var branch1 = new Branch
            {
                BranchName = "Computer Science",
                Description = "Focuses on software development and computing technologies.",
                PhoneNumber = "123-456-7890",
                Email = "cs@dotnettutorials.net"
            };
            var branch2 = new Branch
```

```

    {
        BranchName = "Electrical Engineering",
        Description = "Focuses on electrical systems and circuit
design.",
        PhoneNumber = "987-654-3210",
        Email = "ee@dotnettutorials.net"
    };
    // Add the branches to the context
    context.Branches.Add(branch1);
    context.Branches.Add(branch2);
    // Save changes to the database
    context.SaveChanges();
    Console.WriteLine("Branches added successfully!");
}
private static void AddStudents(EFCoreDbContext context)
{
    // Retrieve the branches from the context
    var csBranch = context.Branches.FirstOrDefault(b => b.BranchName
== "Computer Science");
    var eeBranch = context.Branches.FirstOrDefault(b => b.BranchName
== "Electrical Engineering");
    // Create two new Student objects
    var student1 = new Student
    {
        FirstName = "Pranaya",
        LastName = "Rout",
        DateOfBirth = new DateTime(2000, 1, 15),
        Gender = "Female",
        Email = "Pranaya.Rout@dotnettutorials.net",
        PhoneNumber = "555-1234",
        EnrollmentDate = DateTime.Now,
        Branch = csBranch // Assign the Computer Science branch
    };
    var student2 = new Student
    {
        FirstName = "Rakesh",
        LastName = "Kumar",
        DateOfBirth = new DateTime(1999, 10, 22),
        Gender = "Male",
        Email = "Rakesh.Kumar@dotnettutorials.net",
        PhoneNumber = "555-5678",
        EnrollmentDate = DateTime.Now,
        Branch = eeBranch // Assign the Electrical Engineering branch
    };
    // Add the students to the context
    context.Students.Add(student1);
    context.Students.Add(student2);
    // Save changes to the database
    context.SaveChanges();
    Console.WriteLine("Students added successfully!");
}

```

```

private static void GetAllStudents(EFCoreDbContext context)
{
    // Retrieve all students from the context
    var students = context.Students.Include(s => s.Branch).ToList();
    // Display the students in the console
    Console.WriteLine("All Students:");
    foreach (var student in students)
    {
        Console.WriteLine($"{student.StudentId}:
{student.FirstName} {student.LastName}, Branch:
{student.Branch?.BranchName}");
    }
}

private static void GetStudentById(EFCoreDbContext context, int
studentId)
{
    // Retrieve a single student by ID
    var student = context.Students.Include(s =>
s.Branch).FirstOrDefault(s => s.StudentId == studentId);
    if (student != null)
    {
        Console.WriteLine($"Student found: {student.FirstName}
{student.LastName}, Branch: {student.Branch?.BranchName}");
    }
    else
    {
        Console.WriteLine($"Student with ID {studentId} not found.");
    }
}

private static void UpdateStudent(EFCoreDbContext context, int
studentId)
{
    // Retrieve the student from the context
    var student = context.Students.FirstOrDefault(s => s.StudentId ==
studentId);
    if (student != null)
    {
        // Update the student's information
        student.LastName = "UpdatedLastName";
        student.Email = "updated.email@dotnettutorials.net";
        // Save changes to the database
        context.SaveChanges();
        Console.WriteLine($"Student with ID {studentId} updated
successfully.");
    }
    else
    {
        Console.WriteLine($"Student with ID {studentId} not found.");
    }
}

```



```

        private static void DeleteStudent(EFCoreDbContext context, int
studentId)
        {
            // Retrieve the student from the context
            var student = context.Students.FirstOrDefault(s => s.StudentId ==
studentId);
            if (student != null)
            {
                // Remove the student from the context
                context.Students.Remove(student);
                // Save changes to the database
                context.SaveChanges();
                Console.WriteLine($"Student with ID {studentId} deleted
successfully.");
            }
            else
            {
                Console.WriteLine($"Student with ID {studentId} not found.");
            }
        }
    }
}

```

Now, run the application, and you should see the following output.

```

Branches added successfully!
Students added successfully!
All Students:
    1: Pranaya Rout, Branch: Computer Science
    2: Rakesh Kumar, Branch: Electrical Engineering
Student found: Pranaya Rout, Branch: Computer Science
Student with ID 1 updated successfully.
Student with ID 2 deleted successfully.
All Students:
    1: Pranaya UpdatedLastName, Branch: Computer Science
All operations completed successfully!

```

Note: In Entity Framework Core (EF Core), SQL queries are generated and executed at runtime, not at compilation time. EF Core dynamically constructs the SQL queries when the LINQ query is actually executed at runtime.

What is the Problem of Hardcoding the Connection String?

Hardcoding the database connection string is a bad programming practice. In real-time applications, we need to store the connection string in the appsettings.json file or any secure channel, and from the configuration file, we need to read the connection string. But by default, the **appsettings.json** file is not available in the Console Application. So, let us proceed and understand the steps required to add and use the appsettings.json file in the Console Application,

How Do We Use AppSettings.json Files in .NET Core Console Application?

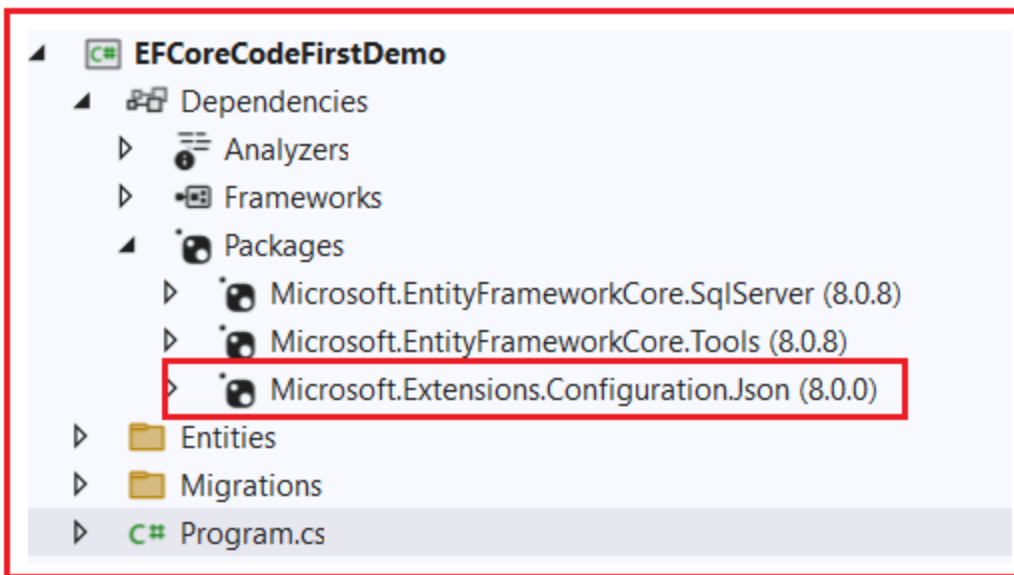
For the .NET Framework Console Application, we always use the app.config file to store our configuration values for the application, such as Connection Strings, Application Level Global Variables, etc. In .NET Core, instead of App.Config file: we need to use the appsettings.json file.

This appsettings.json file is available by default in ASP.NET Core Web Applications such as ASP.NET Core MVC and Web API. However, it is not available by default for the Console Application. Let us proceed and see how we can create and use the appsettings.json file in the .NET Core Console Application.

To use the **appsettings.json** file in the Console Application, we need to install the **Microsoft.Extensions.Configuration.Json** package from NuGet using either the NuGet Package Manager UI or the Package Manager Console. So, open Package Manager Console and then execute the following command:

Install-package Microsoft.Extensions.Configuration.Json

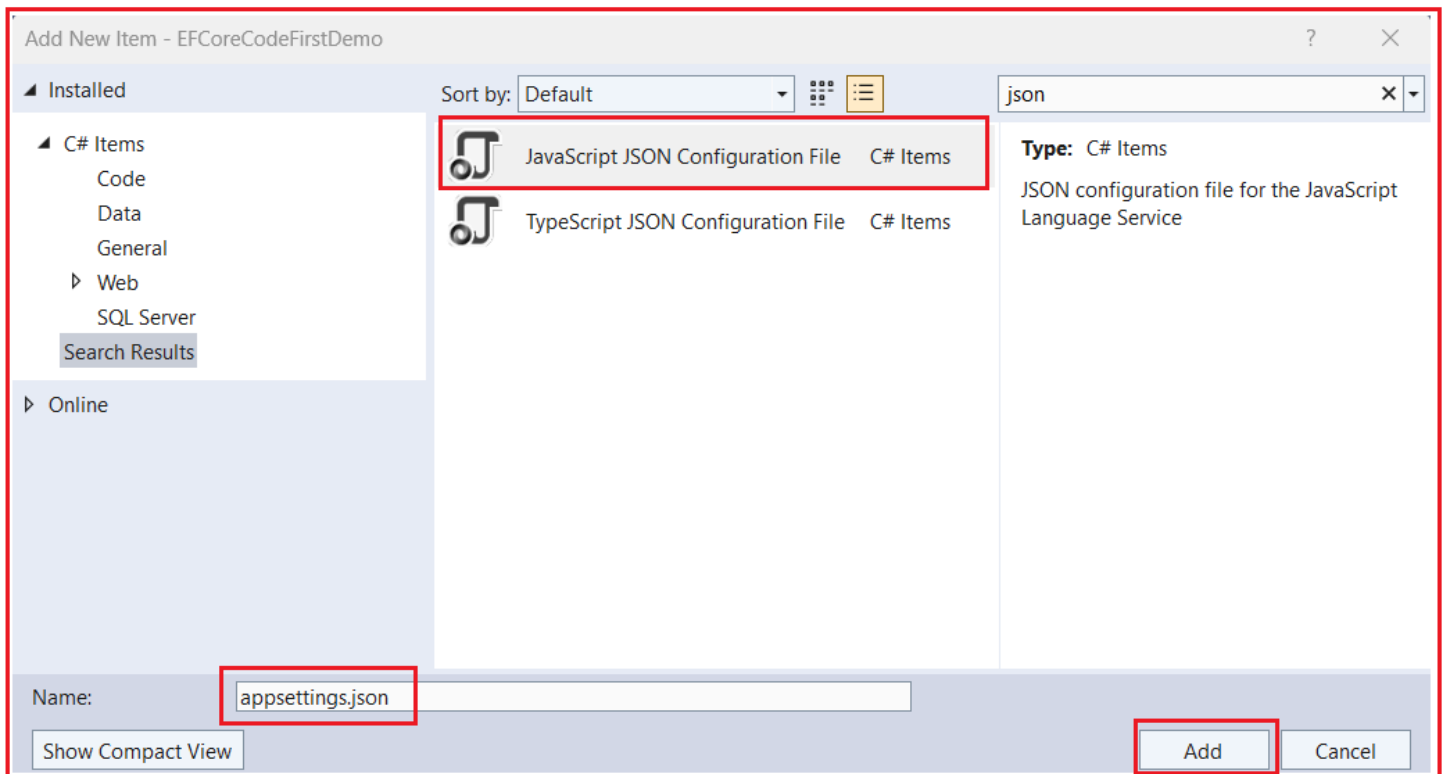
Once you execute the above command, you can verify the package inside your project's **Dependencies => Packages** folder, as shown in the image below.



Adding appsettings.json File:

Once you install the Microsoft.Extensions.Configuration.Json package, the next step is to add a JSON file named **appsettings.json** to the project root directory. While the name does not always need to be appsettings, this is a naming convention we generally follow in .NET Core Applications.

So, right-click on your project and select **Add => New Item** from the context menu to open the following Add New Item window. Here, search for JSON and then select JavaScript JSON Configuration File. Provide the file name as **appsettings.json**, and click on the add button, which will add **the appsettings.json** file to the root directory of your project.



Once you add the **appsettings.json** file, please open it and copy and paste the following code. Here, we are adding the database connection string. So, here we are creating a section called **ConnectionStrings**, and inside this section, we add a key named **SQLServerConnection**, which holds the value of our database connection string.

```
{
  // "ConnectionStrings" is a section that holds one or more database connections
  // It's a common convention in .NET applications to store connection strings in this section
  "ConnectionStrings": {
    // "SQLServerConnection" is the key used to identify this particular connection
    // The value associated with this key is the actual connection string used to connect to the
    Server database.
    // The connection string contains several key-value pairs separated by semicolons
    // 1. "Server": Specifies the name of the SQL Server instance to connect to
    //    - Here, "LAPTOP-6P5NK25R\\SQLSERVER2022DEV" indicates the server name and instance
    // 2. "Database": The name of the database to connect to, in this case, "EFCoreDB1"
    // 3. "Trusted_Connection=True": Indicates that Windows Authentication is used to connect to
    the database.
    // 4. "TrustServerCertificate=True": Bypasses the certificate trust chain validation (use with
    caution in production).
    "SQLServerConnection": "Server=LAPTOP-6P5NK25R\\SQLSERVER2022DEV;Database=EFCoreDB1;Trusted_Connection=True;TrustServerCertificate=True"
  }
}
```

Loading the Connection String From the appsettings.json file:

Now, we need to fetch the connection string from the appsettings.json file. So, modify the **EFCoreDbContext** class as follows. The steps to load the connection string from the appsettings.json file are explained through the comment lines.

```
using Microsoft.EntityFrameworkCore;
```

```

using Microsoft.Extensions.Configuration;
namespace EFCoreCodeFirstDemo.Entities
{
    // EFCoreDbContext is your custom DbContext class that extends the base
    DbContext class provided by EF Core.
    public class EFCoreDbContext : DbContext
    {
        // The OnConfiguring method allows us to configure the DbContext
options,
        // such as specifying the database provider and connection string.
        protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
        {
            // Step 1: Load the Configuration File (appsettings.json).
            // The ConfigurationBuilder class is used to construct
configuration settings from various sources.
            // Here, we add the appsettings.json file to the configuration
sources and then build it.
            var configBuilder = new ConfigurationBuilder()
                .AddJsonFile("appsettings.json") // Specify the configuration
file to load.
                .Build(); // Build the configuration object, making it ready
to retrieve values.
            // Step 2: Get the "ConnectionStrings" section from the
configuration.
            // The GetSection method is used to access a specific section
within the configuration file.
            // Here, we are accessing the "ConnectionStrings" section which
contains our database connection strings.
            var configSection =
configBuilder.GetSection("ConnectionStrings");
            // Step 3: Retrieve the connection string value using its key
("SQLServerConnection").
            // The indexer [] is used to access the value corresponding to
the "SQLServerConnection" key within the section.
            // The null-coalescing operator (??) ensures that if the key is
not found, it will return null.
            var connectionString = configSection["SQLServerConnection"] ??
null;
            // Step 4: Configure the DbContext to use SQL Server with the
retrieved connection string.
            // The UseSqlServer method is an extension method that configures
the context to connect to a SQL Server database.
            optionsBuilder.UseSqlServer(connectionString);
        }
        // DbSet<Student> corresponds to the Students table in the database.
        // It allows EF Core to track and manage Student entities.
        public DbSet<Student> Students { get; set; }
        // DbSet<Branch> corresponds to the Branches table in the database.
        // It allows EF Core to track and manage Branch entities.
        public DbSet<Branch> Branches { get; set; }
    }
}

```

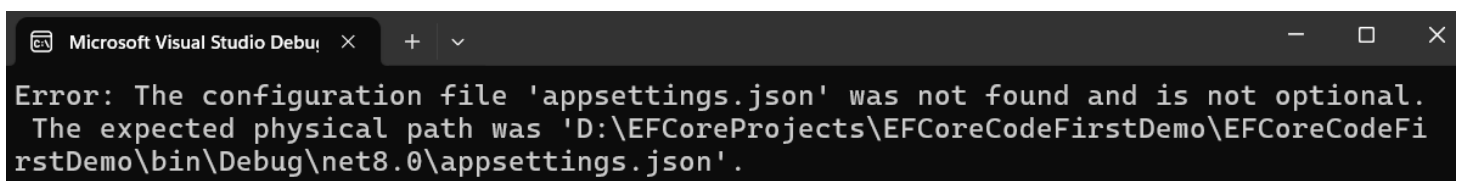
```
}  
}
```

Modifying the Program Class:

Next, modify the Program class as shown below. Now, we are only fetching and displaying all students in the Console.

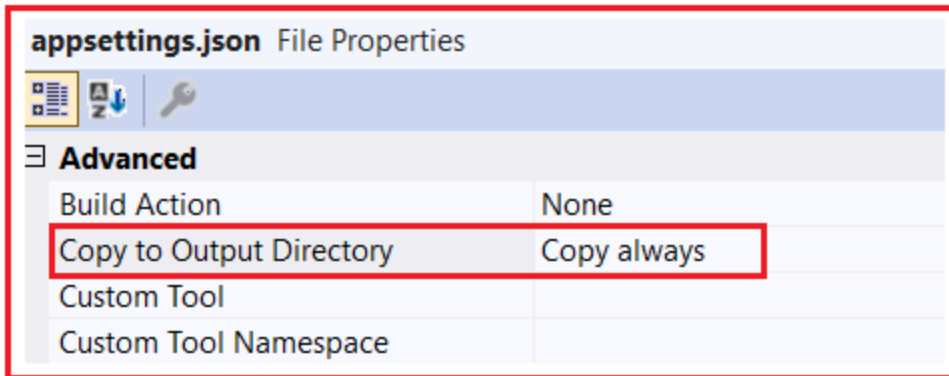
```
using EFCoreCodeFirstDemo.Entities;  
using Microsoft.EntityFrameworkCore;  
namespace EFCoreCodeFirstDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            try  
            {  
                // Create an instance of the DbContext class  
                using var context = new EFCoreDbContext();  
                GetAllStudents(context);  
            }  
            catch (Exception ex)  
            {  
                Console.WriteLine($"Error: {ex.Message}"); ;  
            }  
        }  
        private static void GetAllStudents(EFCoreDbContext context)  
        {  
            // Retrieve all students from the context  
            var students = context.Students.Include(s => s.Branch).ToList();  
            // Display the students in the console  
            Console.WriteLine("All Students:");  
            foreach (var student in students)  
            {  
                Console.WriteLine($"{student.StudentId}:  
{student.FirstName} {student.LastName}, Branch:  
{student.Branch?.BranchName}");  
            }  
        }  
    }  
}
```

With the above changes in place, now run the application, and you should get the following Runtime Exception.



Why are we getting the above Exception?

The above exception clearly says that the appsettings.json file was not found inside the project **bin=>Debug=>.net8.0** folder. That means we need to ensure that once we build the project, the **appsettings.json** file is stored inside the above location. To do so, Right-click the **appsettings.json** file and click on the Properties option to open the following properties window. Then, set its “**Copy to Output Directory**” property to “**Copy always**”. This ensures that the appsettings.json file is copied to the Debug or Release folder every time we build the project.



So, with the above changes in place, build the project and run the application. You should get the output as expected, as shown in the image below.

```
All Students:  
1: Pranaya UpdatedLastName, Branch: Computer Science
```

In the next article, I will discuss [CRUD Operations in Entity Framework Core \(EF Core\)](#). In this article, I explain **How to Use the Database Connection String in Entity Framework Core and Perform Database Operations**. I hope you enjoy this Database Connection String in Entity Framework Core article.